Format commun d'événement

Proposé par Elias (besoins du modèle), **validé côté API** en séance 2 par Fati, qui a repris les cartes de Yarkin (voir `docs/api.md`). L'API Node.js vérifie chaque événement reçu avec ce format (`backend/src/validation.js`) et le stocke dans la table MySQL `evenements` (`backend/db/init.sql`).

Tous les composants parlent ce format : le proxy le produit, l'API le reçoit (`POST /events`), la base le stocke et le tableau de bord l'affiche. Un événement = **une décision** prise par le proxy sur une requête.

1. Exemple

```json
{
  "id": "6f1c2b0e-8a4d-4c1e-9f3a-2d7b5e0c9a11",
  "date": "2026-11-12T10:42:07Z",
  "source": {
    "poste": "pc-test-01",
    "module": "M1"
  },
  "site_ia": "chatgpt.com",
  "type_donnee": "IBAN",
  "regle": "m1.iban",
  "detecteur": "regle",
  "score": 1.0,
  "etiquette": "finance",
  "explication": "IBAN français valide (clé de contrôle correcte) détecté dans le prompt.",
  "action": "bloquer",
  "extrait_masque": "…virer 1 200 € sur FR76 **** **** **** **** **** 189 avant…",
  "version_modele": null,
  "latence_ms": 4
}
```

2. Les champs

Champ	Type	Obligatoire	Description
`id`	UUID (texte)	oui	Identifiant unique, généré par le proxy
`requete_id`	UUID (texte)	non	Commun à tous les événements d'une même requête (voir section 5)
`date`	ISO 8601, UTC	oui	Moment de la détection
`source.poste`	texte	oui	Nom du PC de test (jamais le nom d'une personne)
`source.module`	`M1` / `M2` / `M3`	oui	Module qui a produit l'événement
`site_ia`	texte	oui	Domaine visé : `chatgpt.com`, `gemini.google.com`, `pypi.org` …
`type_donnee`	texte	oui	Type précis : `IBAN`, `CARTE`, `NIR`, `SIRET`, `TELEPHONE`, `EMAIL`, `CLE_API`, `MOT_DE_PASSE`, `PAQUET_SUSPECT` …

Champ	Type	Obligatoire	Description
`regle`	texte	oui	Identifiant de la règle dans `rules/m1.yaml` ou `rules/m2.yaml`, ou `modele` si c'est le classifieur seul
`detecteur`	`regle` / `modele` / `les_deux`	oui	Qui a détecté
`score`	nombre de 0 à 1	oui	Confiance. Une règle validée vaut `1.0`, le modèle donne sa probabilité
`etiquette`	une des 6 étiquettes du guide	oui (M1)	`non_sensible`, `identite`, `finance`, `sante`, `secret_technique`, `donnees_clients`
`explication`	texte en français (≤ 500 car.)	oui	Pourquoi l'alerte : affichée telle quelle dans le tableau de bord
`action`	`bloquer` / `masquer` / `alerter` / `laisser`	oui	Décision appliquée par le proxy
`extrait_masque`	texte (≤ 200 car.)	oui	Contexte autour de la donnée, **avec la donnée masquée**
`version_modele`	texte ou `null`	non	Version du classifieur utilisé, ex. `baseline-tfidf-v0`
`latence_ms`	entier	non	Temps d'analyse du prompt par le proxy

3. Pourquoi ces champs (besoins du modèle)

- `score` **toujours entre 0 et 1** : on peut trier et filtrer règles et modèle sur la même échelle, et régler un seuil d'alerte depuis la page de réglages.
- `detecteur` : on compare ce que les règles attrapent seules et ce que seul le modèle attrape, comme le prévoit le travail d'Imad.
- `etiquette` : mêmes valeurs que le jeu de données. Les statistiques du tableau de bord se calculent directement par catégorie.
- `version_modele` : quand on réentraîne le modèle, on sait quelle version a produit quelle alerte.
- `latence_ms` : sert à la métrique « temps de réponse par prompt ».
- `explication` : sert de base au module bonus 4 (alertes expliquées).

4. Règles RGPD sur le format

- **Jamais la donnée sensible en clair** : ni dans `extrait_masque`, ni ailleurs. Le masquage se fait **dans le proxy, avant l'envoi** à l'API.
- **Jamais le prompt complet** : seulement un extrait de 200 caractères au maximum.
- **Pas de nom de personne** dans `source.poste`.

Règle de masquage proposée : on garde les 4 premiers et les 3 derniers caractères, et on remplace le reste par `*` (pour un e-mail : `j****@e******.fr`). Pour les clés API et les mots de passe, **tout** est masqué.

5. Plusieurs données dans un même prompt

Un prompt qui contient 2 données produit **2 événements** (un par donnée), avec le même champ `requete_id`. L'action appliquée à la requête est la plus forte : `bloquer` > `masquer` > `alerter` > `laisser`.

6. Décisions prises côté API (séance 2)

Tranchées par Fati en construisant l'API (détail dans `docs/api.md`) :

Question	Décision
Noms des champs : français ou anglais ?	**Français**, exactement comme dans ce document
Ajouter `requete_id` ?	**Oui**, facultatif : un UUID commun aux événements d'une même requête
`source.module` : champ à part ou déduit de `regle` ?	**Champ à part**, obligatoire (`M1`, `M2` ou `M3`)
Table : une colonne par champ ou JSON ?	**Une colonne par champ** dans la table MySQL `evenements`

L'API ajoute 3 champs de suivi qui ne viennent pas du proxy : `statut` (`nouveau` par défaut), `commentaire` et `recu_le`.

7. Lien avec `detect()`

`detect()` renvoie `categorie` et `source` (`docs/detect.md`), alors que l'événement attend `etiquette` et `detecteur`. L'addon fait la conversion : la version 1 d'Arno (`proxy/cybersentinel.py`, branche `cyber`) remplit `etiquette`, `detecteur` et `requete_id` à partir de la sortie de `detect()`.
